

The Table (Hash Table) Design

COOL Compiler — `common/table.h` / `table.c`

Why use a table (hash table) at all?

The compiler repeatedly needs to answer one question, for identifiers, string literals, and integer literals alike: "have I already seen this exact piece of text?" If it has, every later phase should get back the same handle for it, so comparisons become a pointer/identity check instead of a character-by-character string comparison repeated at every phase (parsing, semantic analysis, codegen). A hash table answers this in $O(1)$ average time. A flat list or repeated linear search would degrade to $O(n)$ per lookup, which on a real program with thousands of identifier occurrences becomes the dominant cost of compilation.

This single operation — intern-or-fetch — is used identically by three different kinds of text (identifiers, string constants, integer constants), so one generic table implementation is written once and reused three times, rather than three separate hand-rolled lookup structures.

Specification of the table

The table is designed as a strict black box: every caller outside `table.c` only ever calls its published API (`table_create`, `table_create_sized`, `table_add_entry`, `table_destroy`, and the `entry_*` accessors). No caller reaches into `buckets`, the hash function, or the bucket count directly. This means the table alone decides and owns everything about *how* it achieves fast lookups — bucket count, hash function, collision strategy, growth policy — and that internal implementation can change later (e.g. adding automatic resizing) without any caller needing to change.

- Guarantees near $O(1)$ average-case lookup and insertion, regardless of how many entries a given COOL source file produces.
- Guarantees a stable identity for every distinct piece of text: the same text always returns the same `Entry*`, for the lifetime of the table.
- Owns its own memory fully: every entry, and the string copy inside it, is allocated by the table and freed by the table, never partially by a caller.
- Exposes a uniform interface regardless of what is being interned — the same function signature (`table_add_entry`) works for identifiers, strings, and integers, differing only in the size of the entry struct passed in.

What entries the table stores

Every table stores a chain of `Entry` structs per bucket. The base `Entry` carries exactly four fields, each earning its place for a specific downstream need:

Field	Purpose
str	Heap-owned copy of the interned text. Not a borrowed pointer into the source buffer — the table must outlive the raw source buffer, so it owns its own copy.
len	Length in bytes. Never rely on a null terminator alone, since interning happens on slices of the source that are not naturally null-terminated at scan time.
index	Creation order within this table. Used only much later, by code generation, to produce a unique assembly label per entry (e.g. str_const7).
next	Collision-chain pointer to the next entry in the same bucket.

Callers that need extra per-entry metadata (see [interntables.pdf](#) for `StringEntry` / `IntEntry`) embed this base `Entry` as the first field of a larger struct, rather than the table needing to know about that extra data.

Who owns the table

Ownership follows the lifetime the table actually needs: it must exist before the lexer produces the first token, and it must still exist when code generation walks every string/int entry at the very end of compilation. No single phase (lexer, parser, semantic analysis, codegen) has that lifespan on its own — only the driver does.

- `main.c` (the driver) creates the tables once, before lexing begins, and destroys them once, after the entire pipeline has finished.
- Every phase receives a pointer to the tables it needs and only ever reads/writes through the published API — no phase allocates or frees a `Table` itself.
- This mirrors the same ownership discipline used everywhere else in the compiler: whoever creates a resource is responsible for destroying it, and that responsibility sits at the narrowest scope that still covers the resource's full required lifetime.

Implementation technique: separate chaining

The table is implemented as a hash table using **separate chaining**: `buckets` is an array of `Entry*`, and each bucket is the head of a singly linked list of entries that hashed to the same index. This was chosen deliberately over open addressing (linear/quadratic probing):

- **Unbounded entry count.** A COOL source file may contain any number of identifiers; chaining lets the table keep accepting entries gracefully (longer chains) rather than requiring the backing array itself to be large enough to hold every entry, the way open addressing does.
- **Pointer stability.** Every `Entry` is its own independent heap allocation that never moves once created. This is essential because the rest of the compiler holds and compares raw `Entry*` pointers for the entire compilation — open addressing stores entries inside the array itself, so any future resize/rehash would silently invalidate every pointer anyone else is holding.

- **Simplicity of deletion** (not currently used, but kept cheap for future reuse): unlinking a node from a chain is trivial; open addressing requires tombstones and periodic cleanup to delete safely.

Hashing function: djb2

The table uses **djb2** (Dan Bernstein's hash):

```
unsigned long hash(const char *s, int len) {
    unsigned long h = 5381;
    for (int i = 0; i < len; i++)
        h = ((h << 5) + h) + (unsigned char) s[i]; /* h * 33 + c */
    return h;
}
```

- **Well suited to short strings.** Identifiers, keywords, and small literal constants are exactly the kind of short ASCII text djb2 was empirically shown to distribute well.
- **Fast.** Two operations per character (shift-and-add, then add the character) — cheap enough to run on every single character the lexer scans.
- **Deterministic.** Same input always produces the same hash, which is required — the compiler is not processing adversarial/attacker-controlled input in a way that would call for a randomized hash the way, say, a public-facing hash map implementation might.
- The alternative considered was FNV-1a, which is roughly equivalent in cost and quality for this workload; djb2 was chosen as the simpler, more commonly taught, well-validated default for exactly this kind of interning table.

Bucket count

A table is initialized with `num_buckets = 2 * expected_entries + 1`. The default entry point, `table_create()`, assumes an expected size of 105, yielding 211 buckets; callers that know their expected size in advance (as `InternTables` does) can call `table_create_sized` directly.

- **Odd/near-prime sizing.** $2n + 1$ always produces an odd number, which helps spread hashed values more evenly across buckets than an arbitrary even number would, reducing the chance that a common factor between the hash output and the bucket count causes systematic clustering.
- **Sized ahead, not grown reactively.** Bucket count is decided once, at creation, from an expected entry count, rather than starting tiny and relying on repeated resizing during scanning. For a single COOL source file (typically a few hundred identifiers), an upfront reasonable size avoids the cost and complexity of a resize-and-rehash path, while still keeping average chain length short.
- **Decoupled from the actual entry count going in.** Because chaining is used, exceeding the initial bucket count does not fail or require an immediate resize — it simply means average chain length grows gradually, which is an acceptable, graceful degradation rather than a hard limit.

Other specific implementation decisions worth noting

- `table_add_entry` **takes an explicit `entry_size` parameter** rather than hardcoding `sizeof(Entry)`. This is what allows callers to intern into a larger struct (`StringEntry`, `IntEntry`) that embeds `Entry` as its first field, without the table needing any awareness of those derived types.

- A `bool *was_new` **out-parameter** distinguishes a freshly created entry from one that already existed. This matters because a newly created entry's extra fields (on `StringEntry/IntEntry`) must be initialized exactly once; re-initializing them on every repeated interning hit would silently discard state a later phase (codegen) may have already computed.
- buckets **is zero-initialized with** `calloc`, **not** `malloc` — every bucket must start as a guaranteed NULL head, since the lookup/insert loop assumes an empty bucket is exactly that.
- Every allocation on the hot path (`malloc/calloc/realloc`) is checked for NULL and fails loudly via `perror + exit(1)` rather than silently continuing with a bad pointer.
- The table exposes **no direct field access** to callers — `entry_str`, `entry_len`, `entry_index` are the only sanctioned ways to read an entry's data, keeping the struct layout free to change internally without breaking callers.